

Ayudantía 3

Recursividad y Complejidad Computacional

Enzo Vásquez C.

Escuela de Ingeniería
Universidad de O'Higgins

25/9/2026



- ① Recursividad
- ② Complejidad Computacional
- ③ Ejercicios

1 Recursividad

2 Complejidad Computacional

3 Ejercicios

¿Qué es la recursividad?

- Técnica donde una función se llama a sí misma para resolver un problema.

¿Qué es la recursividad?

- Técnica donde una función se llama a sí misma para resolver un problema.
- Consiste en reducir el problema actual a una versión más pequeña.

¿Qué es la recursividad?

- Técnica donde una función se llama a sí misma para resolver un problema.
- Consiste en reducir el problema actual a una versión más pequeña.
- **Tres componentes indispensables:**

¿Qué es la recursividad?

- Técnica donde una función se llama a sí misma para resolver un problema.
- Consiste en reducir el problema actual a una versión más pequeña.
- **Tres componentes indispensables:**
 - ① **Caso base:** Situación simple, sin otra llamada. Indica el fin.

¿Qué es la recursividad?

- Técnica donde una función se llama a sí misma para resolver un problema.
- Consiste en reducir el problema actual a una versión más pequeña.
- **Tres componentes indispensables:**
 - ① **Caso base:** Situación simple, sin otra llamada. Indica el fin.
 - ② **Llamado recursivo:** Expresa la solución usando la misma función.

¿Qué es la recursividad?

- Técnica donde una función se llama a sí misma para resolver un problema.
- Consiste en reducir el problema actual a una versión más pequeña.
- **Tres componentes indispensables:**
 - ① **Caso base:** Situación simple, sin otra llamada. Indica el fin.
 - ② **Llamado recursivo:** Expresa la solución usando la misma función.
 - ③ **Progreso:** Cada llamada se acerca al caso base.

Ejemplos de Recursividad

1. Factorial de un número

```
1 def factorial(n):  
2     if n <= 1:  
3         return 1  
4     return n * factorial(n - 1)
```

2. Suma de una lista

```
1 def sumar(valores):  
2     if not valores:  
3         return 0  
4     primero = valores[0]  
5     resto = valores[1:]  
6     return primero + sumar(resto)
```

Las Torres de Hanói

```
1 def hanoi(n, A, B, C):
2     if n == 1:
3         disco = A.pop()
4         C.append(disco)
5         return [disco]
6
7     mover = hanoi(n - 1, A, C, B)
8
9     disco = A.pop()
10    C.append(disco)
11    mover.append(disco)
12
13    return mover + hanoi(n - 1, B, A, C)
```

- 1 Recursividad
- 2 Complejidad Computacional
- 3 Ejercicios

Eficiencia y Notación Big $\mathcal{O}$

- No medimos segundos exactos, medimos cómo escala el costo cuando n **crece**.
- Big $\mathcal{O}$ denota la cota asintótica superior, en otras palabras, cuanto tarda **a lo más** el algoritmo en ejecutarse.

$$\underbrace{\mathcal{O}(1) \subset \mathcal{O}(\log n) \subset \mathcal{O}(n) \subset \mathcal{O}(n \log n) \subset \mathcal{O}(n^k)}_{\text{Tratables}} \subset \underbrace{\mathcal{O}(c^n) \subset \mathcal{O}(n!) \subset \mathcal{O}(n^n) \subset \dots}_{\text{Intratables}}$$

1 Recursividad

2 Complejidad Computacional

3 Ejercicios

Problema 1: Detección de fraudes

Problema 2: Organización de Torneos

Problema 3: Logística y Envíos

1 Recursividad

2 Complejidad Computacional

3 Ejercicios

Problema 1: Detección de fraudes

Problema 2: Organización de Torneos

Problema 3: Logística y Envíos

Detección de fraudes bancarios

Un banco tiene un registro histórico de millones de transacciones y una lista negra de operaciones fraudulentas. Debemos verificar si las transacciones del cliente están en la lista.

Analice e implemente las siguientes alternativas:

- **Método A:** Comparar cada transacción del cliente contra toda la lista negra.
- **Método B:** Buscar en lista usando `x in lista`.
- **Método C:** Convertir la lista negra a un set y usar `x in set`.

Detección de fraudes bancarios

Puedo considerar el siguiente fragmento de código para modelar los datos.

```
1 transacciones = list(range(50000))  
2 lista_negra = list(range(45000, 55000))  
3 fraudes = 0  
4  
5 #Implementar método
```

1 Recursividad

2 Complejidad Computacional

3 Ejercicios

Problema 1: Detección de fraudes

Problema 2: Organización de Torneos

Problema 3: Logística y Envíos

Organización de torneos

Un torneo de X juego necesita estructurar su organización de acuerdo al elo de cada jugador. Para ello, se desea ordenar los jugadores de mayor a menor elo.

Misiones a programar:

- 1 **Carga de datos:** Leer el archivo `jugadores.txt`, procesar cada línea e instanciar los objetos `Jugador`, guardándolos en una lista.
- 2 **Ordenamiento manual:** Implementar un algoritmo basado en comparar pares para ordenar la lista de mayor a menor ELO. El costo esperado para este enfoque es de $\mathcal{O}(n^2)$.
- 3 **Ordenamiento build-in:** Utilizar la función nativa de Python (`sorted()`) para realizar el mismo ordenamiento.

1 Recursividad

2 Complejidad Computacional

3 Ejercicios

Problema 1: Detección de fraudes

Problema 2: Organización de Torneos

Problema 3: Logística y Envíos

Logística: Calculando el peso de un envío

En un centro de distribución, un envío puede contener productos sueltos o **cajas**. A su vez, una caja puede contener más productos u otras cajas más pequeñas.

```
1 class Producto:
2     pass
3
4 class Caja:
5     pass
```

Instrucciones del Ejercicio

Implementar:

- 1 **Modelado (POO):** Crear la clase `Producto` (con atributos `nombre` y `peso`) y la clase `Caja` (con el atributo `contenido`, que será una lista de elementos).
- 2 **Función Recursiva:** Implementar `calcular_peso(elemento)`.
- 3 **Pruebas:** Instanciar una caja principal que contenga al menos dos productos sueltos y una caja secundaria (la cual debe contener sus propios productos). Calcular e imprimir el peso total.
- 4 **Análisis de eficiencia:** Determinar la complejidad Big $\mathcal{O}$ del algoritmo asumiendo que n es la cantidad total de productos y cajas en el envío.